

Задание:

Тема: «Система учёта задач (To-Do менеджер) с сохранением в файл».

Контекст:

Студент приходит в отдел, получает полуготовый проект и должен его доработать.

Исходные данные (стартовый код):

Студент получает частично работающее приложение (консольное или простой GUI на Tkinter). В нём есть ошибки и отсутствует часть критического функционала.

```
# todo_app.py (базовый каркас)
```

```
import json
```

```
import os
```

```
TASKS_FILE = "tasks.json"
```

```
def load_tasks():
```

```
    if not os.path.exists(TASKS_FILE):
```

```
        return []
```

```
    with open(TASKS_FILE, "r", encoding="utf-8") as f:
```

```
        return json.load(f)
```

```
def save_tasks(tasks):
```

```
    with open(TASKS_FILE, "w", encoding="utf-8") as f:
```

```
        json.dump(tasks, f, indent=4, ensure_ascii=False)
```

```
def add_task(tasks, title):
```

```
    # TODO: добавить задачу в формате {'id': автоинкремент, 'title': title, 'done': False}
```

```
    pass
```

```
def show_tasks(tasks):  
    # TODO: вывести все задачи красиво с статусом [X] или [ ]  
    pass  
  
def complete_task(tasks, task_id):  
    # TODO: отметить задачу как выполненную (done = True)  
    pass  
  
def delete_task(tasks, task_id):  
    # TODO: удалить задачу по ID  
    pass  
  
# основной цикл  
def main():  
    tasks = load_tasks()  
    while True:  
        print("\n1. Добавить задачу\n2. Показать задачи\n3. Выполнить задачу\n4. Удалить задачу\n5.  
Выход")  
        choice = input("Выберите действие: ")  
        if choice == "1":  
            title = input("Задача: ")  
            add_task(tasks, title)  
            save_tasks(tasks)  
        elif choice == "2":  
            show_tasks(tasks)  
        # ... допишите остальные пункты меню  
  
if __name__ == "__main__":  
    main()
```

Задание №1:

1. Реализовать все функции `add_task`, `show_tasks`, `complete_task`, `delete_task`.
2. Добавить нумерацию ID: при добавлении задачи ID должен быть $\max(\text{существующие_id}) + 1$ (или 1, если список пуст).
3. Добавить пункты меню 3 и 4, чтобы вызывались `complete_task` и `delete_task`.
4. Обработка ошибок: при попытке завершить/удалить задачу с несуществующим ID должно выводиться сообщение "Задача не найдена".

Задание №2:

◆ Дополнение кода (вариативное, 2-3 пункта на выбор):

Выберите, какие функции добавить (минимально 2):

1. Срок выполнения
Добавить поле `due_date` (строка в формате ГГГГ-ММ-ДД). При выводе задач показывать дни до дедлайна или "(просрочено)".
2. Поиск задач
Пункт меню «Найти задачу» – поиск по части названия (регистронезависимо).
3. Сортировка
Показывать задачи в порядке: сначала невыполненные, потом выполненные; внутри каждой группы – по ID.
4. Подсчёт статистики
Пункт меню «Статистика»: общее количество задач, выполненных и невыполненных.
5. Сохранение в два файла
Дублировать данные в `tasks_backup.csv` (формат: `id,title,done,date`).
6. Очистка выполненных
Пункт «Удалить все выполненные задачи».
7. Поддержка нескольких списков
Возможность создать/переключиться между несколькими файлами (например, «Работа», «Личное»).

Задание №3:

Задание по желанию – GUI или Web.

Вариант А (Tkinter):

Переписать приложение на графический интерфейс (окно с Listbox, полем ввода и кнопками Add/Complete/Delete).

Вариант Б (Flask):

Сделать веб-версию:

- GET / – показать все задачи с чекбоксами
- POST /add – добавить
- POST /complete/<id> – отметить
- Данные хранятся в JSON (без БД).

P.s. В данной работе Вы научитесь:

работать с чужим кодом (если сами дополните код),

внимательно читать задание (если не скопируете его в ИИ),

тщательно выбирать что и как нужно дорабатывать (если перечитаете выполненную работу),

и делать правильный отчет (если дочитаете до сего момента).